MDA Audio

Manuales de Referencia Técnica de los Lenguajes DSL para MDAA

Especificaciones de gramáticas CIM y PIM

Fernando Real Cisneros

11 de mayo de 2026

Índice general

1. Introducción **4**

2. Concepto de Máquinas y Relaciones **6**

2.1. Señales de Sonido y Modificación 6

2.2. El Concepto de Máquina 6

2.2.1. Máquinas en el nivel CIM 7

2.2.2. Máquinas en el nivel PIM 7

2.3. Relaciones 7

3. CIM (Computation Independent Model) **10**

3.1. Manual de Referencia Técnica MDA-Audio-CIM-Machine 11

3.2. Bases del Modelo 11

3.2.1. Modelo Raíz (`Document`) 11

3.2.2. Interfaz del Elemento (`Element`) 12

3.2.3. Objeto de Conexión Interna (`SendTo`) 13

3.3. Tipos de Elemento de Audio (`Element`) 13

3.4. Reglas de Validación 14

3.4.1. Reglas de Unicidad 14

3.4.2. Restricciones de Campos 14

3.4.3. Restricciones de Conectividad 15

3.5. Guía de Uso 15

3.6. Manual de Referencia Técnica MDA-Audio-CIM-Relations-Machines 16

3.7. Estructura del Modelo 16

3.7.1. Modelo Raíz (`Document`) 16

3.7.2. Objeto de Relación (`Relation`) 16
3.8. Reglas de Validación 17
3.8.1. Identidad y Formato 17
3.8.2. Restricciones de Texto 17
3.8.3. Interfaces Expuestas y Existencia 17
3.9. Guía de Uso 18

4. PIM (Platform Independent Model) 19

4.1. Manual de Referencia Técnica MDA-Audio-PIM 20
4.2. Bases del Modelo 20
4.2.1. Modelo Raíz (`Model`) 20
4.2.2. Interfaces Base 21
4.3. Definición de Aristas (`Edges`) 22
4.4. Diccionario de Nodos 22
4.4.1. Generadores de Audio (`AudioGeneratorNode`) 22
4.4.2. Modificadores de Parámetro (`ParameterModifierNode`) 24
4.4.3. Modificadores de Sonido / Efectos (`SoundModifierNode`) 25
4.4.4. Nodos de Mezcla y Nivel 27
4.5. Apéndice Técnico de Restricciones 27
4.5.1. Formato de Matriz (`Matrix`) 27
4.6. Manual de Referencia Técnica MDA-Audio-PIM-Relations-Machines 28
4.7. Estructura del Documento 28
4.7.1. Propiedades Raíz 28
4.7.2. Objeto Relation (Relación PIM) 28
4.8. Reglas de Validación 29
4.9. Guía de Uso Táctico 29

5. Generación de Paquetes para IA 31

5.1. Proceso de Montaje 31
5.2. Plantillas y README.md 31

6. Ejecución y fuentes de los DSL **33**

6.1. DSL CIM Machine 33

6.1.1. Comandos Principales 33

6.1.2. Validación de Modelos JSON 33

6.2. DSL CIM Relations 34

6.2.1. Comandos Principales 34

6.2.2. Validación de Modelos JSON 34

6.3. DSL PIM Machine 34

6.3.1. Comandos Principales 34

6.3.2. Validación de Modelos JSON 34

6.4. DSL PIM Relations 35

6.4.1. Validación 35

Capítulo 1

Introducción

En este manual se detallan los lenguajes de dominio específico (*DSLs*) desarrollados para el diseño y generación de sintetizadores de audio. Siguiendo el paradigma de *Model-Driven Architecture* (*MDA*), la documentación se divide en dos niveles fundamentales:

- **CIM** (*Computational Independent Model*): Refleja el diseño de alto nivel. En este nivel se define qué elementos componen una máquina y cómo se relacionan a nivel funcional, sin entrar en detalles de implementación técnica o parámetros específicos de síntesis.
- **PIM** (*Platform Independent Model*): Representa el diseño conceptual detallado. Es una evolución del *CIM* donde los elementos abstractos se convierten en nodos de síntesis concretos (osciladores, filtros, envolventes) con parámetros técnicos definidos, manteniendo la independencia de la plataforma final de ejecución.

Es importante destacar que el diseño en el nivel *PIM* siempre hará referencia a la estructura definida previamente en el nivel *CIM*, garantizando la trazabilidad desde la idea funcional hasta la especificación técnica.

La estructura del manual se organiza de la siguiente manera:

- **Conceptos Fundamentales**: Definición de máquinas y flujos de señales.
- **CIM**: Gramáticas para la definición de la estructura funcional.
- **PIM**: Gramáticas para la especificación técnica de síntesis.
- **Aplicación *MDAA***: Integración con herramientas de *IA*, generación de prompts y archivos *JSON*.
- **Ejecución**: Guía práctica para la compilación y validación de los modelos.

Nota informativa

Se ha prescindido de la generación de modelos *PSM* (Modelo Específico de Plataforma), ya que sería redundante para la generación de código en el caso de lenguajes visuales como *Max* o *Pure-Data*.

Nota importante

El diseño en el nivel *PIM* siempre hará referencia a la estructura definida previamente en el nivel *CIM*, garantizando la trazabilidad desde la idea funcional hasta la especificación más técnicoconceptual.

Recomendación

Debido a la especificidad de la materia, se recomienda consultar el documento disponible en `https://github.com/ferquete/mdaa-uned/blob/main/_DOCUMENTACION/material_complementario/latex/main.pdf`. Este documento, realizado ex profeso para lectores no familiarizados con el dominio que abordamos en este artículo, ofrece una visión general sobre la síntesis de sonido digital y *Pure Data*, el lenguaje utilizado en las pruebas.

Capítulo 2

Concepto de Máquinas y Relaciones

Para comprender la arquitectura del sistema, es necesario definir qué consideramos una *máquina*, término que introduzco tanto en los DSL de CIM como de PIM, y cómo interactúan entre sí y con el exterior.

2.1. Señales de Sonido y Modificación

Antes de entrar en el concepto de máquina, debemos distinguir los dos tipos de señales que fluyen por el sistema:

- **Señal de Sonido**: Es el flujo de audio propiamente dicho (formas de onda). Se encarga de transportar la información que finalmente será audicionada.
- **Señal de Modificación**: Es una señal de control que no se oye directamente, sino que se utiliza para alterar parámetros de otros elementos (por ejemplo, una señal de un *LFO* que modifica la frecuencia de un oscilador).

2.2. El Concepto de Máquina

Una **máquina** es un ente funcional generador de sonido, modificador de sonido (señal de control) o ambos. Está compuesta por elementos internos (generadores o modificadores) que colaboran para cumplir su propósito.

Desde una perspectiva técnica, una máquina recibe señales del exterior y envía al exterior sonido o señales de modificación/control a través de sus puertos habilitados.

2.2.1. Máquinas en el nivel CIM

En el nivel *CIM*, la máquina se ve como una caja negra con elementos abstractos. El objetivo es capturar la intención de lo elicitado como requisito, sin especificaciones técnicas. Como veremos cuando analicemos la aplicación MDAA, en su manual de usuario, en este nivel, aunque la herramienta proporciona un visor gráfico de las máquinas, su definición es puramente textual.

2.2.2. Máquinas en el nivel PIM

En el nivel *PIM*, la máquina se descompone en un grafo de nodos técnicos. Cada nodo es una implementación concreta que procesa señales de audio o modificación siguiendo las reglas establecidas por los tipos de nodos posibles. Como veremos cuando entremos en la definición del DSL de máquina en PIM, el número de nodos posibles es limitado, y en esta etapa de la aplicación MDAA solo cubrimos los nodos esenciales que todo desarrollo de este tipo pudiera precisar.

2.3. Relaciones

Dentro de cada modelo (*CIM* y *PIM*), las máquinas establecen relaciones en su interior, entre los módulos/modos que las componen, o con otras máquinas a través de los puertos especificados. Estas relaciones, a nivel CIM, serán sin restricciones, simplemente establecemos la relación entre requisitos, pero en los modelos PIM sí que ya serán más estrictas, limitándose a lo que establece como reglas de diseño el DSL.

Para entender esto mejor, en las figuras siguientes se muestran los conceptos en el nivel *CIM* y *PIM*, capturando imágenes de uno de los proyectos de prueba generados por la aplicación MDAA:

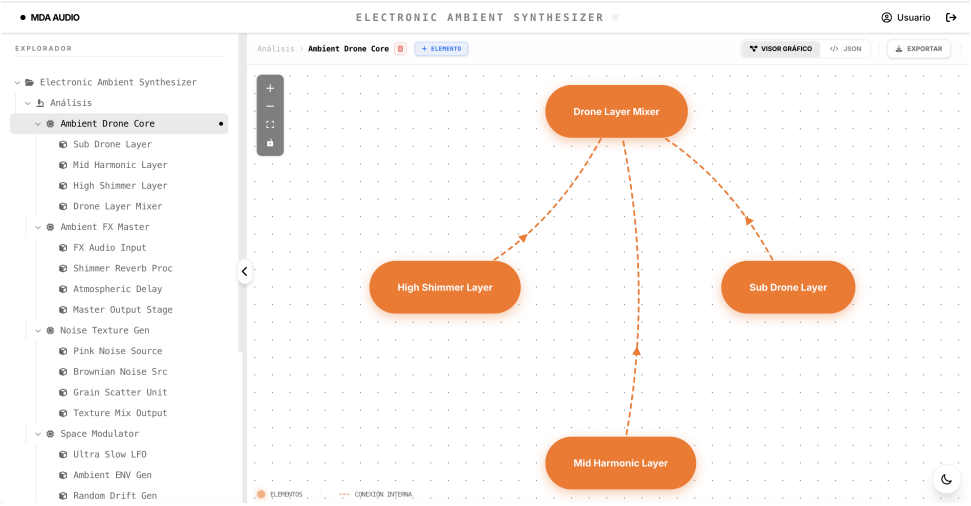

Figura 2.1: Estructura interna de una máquina en el nivel *CIM*.

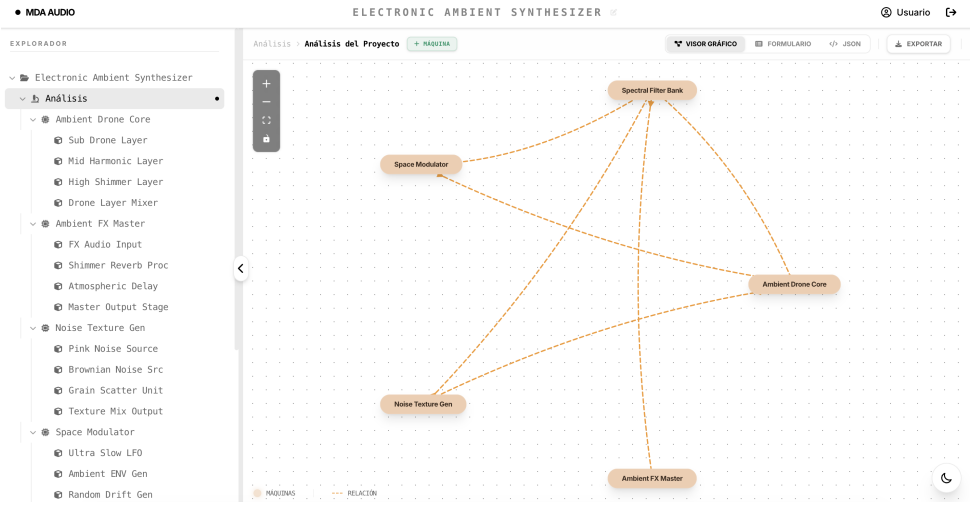

Figura 2.2: Relaciones entre máquinas en el nivel *CIM*.

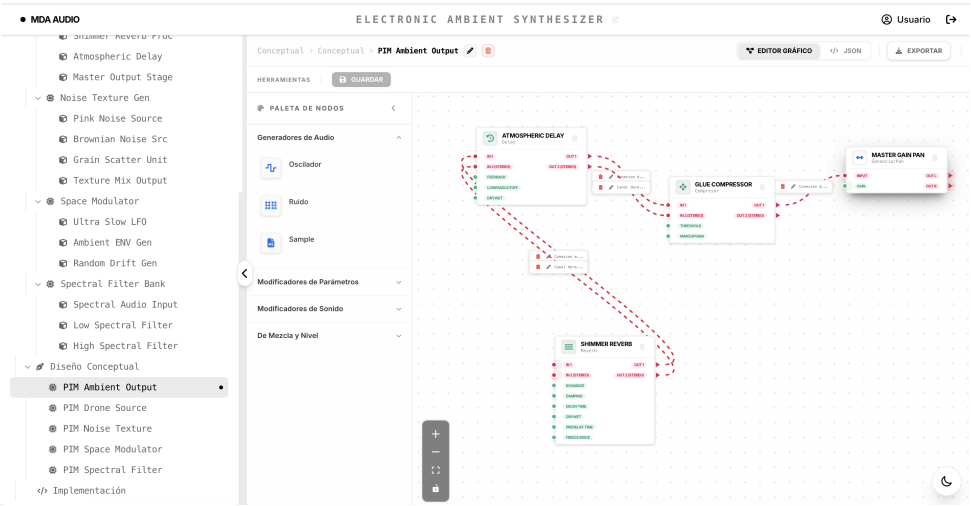

Figura 2.3: Estructura interna de una máquina en el nivel *PIM*.

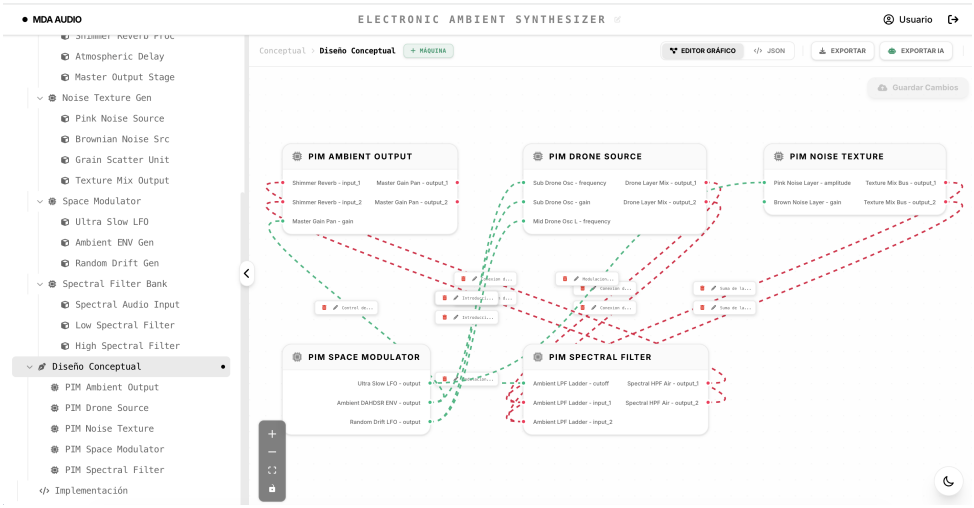

Figura 2.4: Relaciones entre máquinas en el nivel *PIM*.

En las imágenes, ya se puede intuir que en el nivel PIM tenemos una concretización técnica de los conceptos abstractos definidos en el nivel CIM. En PIM ya tenemos una serie de elementos reales pero conceptuales (no concretos de lenguaje o platforma) que se interconectan siguiendo las reglas de diseño establecidas por el DSL.

Capítulo 3

CIM (Computation Independent Model)

3.1. Manual de Referencia Técnica MDA-Audio-CIM-Machine

Este documento es la especificación técnica completa del lenguaje **MDA-Audio-CIM-Machine**. Define la estructura de las máquinas de audio en el nivel más abstracto, así como las relaciones de sus elementos internos entre sí, y las entradas y salidas desde el exterior que pueden recibir o enviar. Siguiendo la nomenclatura de *Model-Driven-Architecture* (MDA), este nivel se conoce como **CIM** (*Computational Independence Model*).

En el CIM no se tiene por qué hablar de implementaciones concretas (osciladores, LFOs, filtros específicos) a bajo nivel, sino de **elementos** abstractos que capturan la intención del diseño de síntesis de la máquina.

3.2. Bases del Modelo

Una máquina es un conjunto de elementos que trabajan juntos para producir sonido. A su vez, como veremos más adelante, las máquinas cooperan entre sí para intercambiar sonido o valores de modificación (ejemplos: volumen, frecuencia, etc.) entre ellas.

3.2.1. Modelo Raíz (`Document`)

El objeto raíz de un archivo MDA-Audio-CIM identifica a la máquina y contiene sus elementos compositivos.

Atributo	Tipo	Descripción	Restricciones
`id`	STRING	Identificador único de la máquina CIM.	Obligatorio. 36 car. (UUID).
`name`	STRING	Nombre descriptivo de la máquina.	Obligatorio. 1 a 20 car.
`description`	STRING	Propósito y contexto de la máquina.	Obligatorio. 10 a 600 car.
`elements`	ARRAY[Element]	Componentes que representan generadores o procesadores.	Obligatorio. Puede estar vacío.

Ejemplo de Documento:

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Oscillator Core",
```

```
  "description": "A fundamental oscillator machine for basic wave generation and shaping.",
  "elements": [ ... ]
}
```

3.2.2. Interfaz del Elemento (`Element`)

Todos los bloques funcionales se definen como `Element`. Todos los campos base son obligatorios.

Atributo	Tipo	Descripción	Restricciones
`id`	STRING	Identificador único del componente.	Obligatorio. 36 car. (UUID).
`name`	STRING	Nombre del elemento.	Obligatorio. 1 a 20 car.
`description`	STRING	Descripción funcional.	Obligatorio. 10 a 600 car.
`params`	STRING	Parámetros modificables.	Obligatorio. 10 a 600 car.
`externalOutput`	ExternalOutput	Salida externa.	Obligatorio.
`externalInput`	ExternalInput	Entrada externa.	Obligatorio.
`sendTo`	ARRAY[SendTo]	Conexiones internas.	Obligatorio. Puede estar vacío.

Ejemplo de Elemento:

```
{
  "id": "771e8400-e29b-41d4-a716-446655440111",
  "name": "Sine Generator",
  "description": "Generates a pure sine wave at a fixed base frequency.",
  "params": "frequency: 440Hz, gain: 0.8, phase: 0.0",
  "externalInput": { "hasExternalInput": false },
  "externalOutput": {
    "hasExternalOutput": true,
    "description": "Audio output of the sine wave."
  },
  "sendTo": []
}
```

Configuración Externa

Los subobjetos `externalOutput` y `externalInput` definen si el elemento sirve como uno de los puertos de entrada/salida global de la máquina.

ExternalOutput / ExternalInput:

- `hasExternalOutput` / `hasExternalInput` (BOOLEAN): Habilita el puerto.
- `description` (STRING): Describe la naturaleza del puerto. Máximo 600 caracteres.

3.2.3. Objeto de Conexión Interna (`SendTo`)

Define un vínculo direccional desde un elemento hacia otro dentro de la misma máquina. El elemento referenciado en el campo `idRef` debe existir dentro del documento.

Atributo	Tipo	Descripción	Restricciones
`id`	STRING	Identificador único de esta conexión.	Obligatorio. 36 car. (UUID).
`idRef`	REF[Element]	Referencia al elemento de destino.	Obligatorio. Debe existir.
`description`	STRING	Justificación técnica del vínculo.	Obligatorio. 10 a 600 car.

Ejemplo de Conexión:

```
{
  "id": "b22e8400-e29b-41d4-a716-446655440333",
  "idRef": "c33e8400-e29b-41d4-a716-446655440444",
  "description": "Route filtered signal to the high pass
   filter element."
}
```

IMPORTANTE

Un elemento **no puede referenciarse a sí mismo** en su lista `sendTo`. El compilador rechaza cualquier auto-referencia como error fatal.

3.3. Tipos de Elemento de Audio (`Element`)

Representan cualquier bloque funcional de señal sonora dentro de la máquina. A este nivel no se especifica el algoritmo de síntesis: eso corresponde al PIM.

Los elementos típicos abarcan:

- **Fuentes / Generadores:** Osciladores, samplers, generadores de ruido.
- **Modificadores / Efectos:** Filtros, reverbs, delays, envolventes ADSR, etc.

El campo `params` describe en lenguaje natural los parámetros conceptuales del elemento (ej.: *"Frequency, Waveform"* o *"Cutoff, Resonance"*).

3.4. Reglas de Validación

Se aplican las siguientes restricciones para garantizar que el modelo sea procesable:

3.4.1. Reglas de Unicidad

Alcance	Regla
Global	Todos los `id` de todos los componentes deben ser únicos en el documento entero.
Local	No puede haber dos entradas en el array `sendTo` de un mismo componente con el mismo `id`.

3.4.2. Restricciones de Campos

Entidad	Campo	Restricción
`Document`	`id`	Exactamente 36 caracteres.
`Document`	`name`	Entre 1 y 20 caracteres.
`Document`	`description`	Entre 10 y 600 caracteres.
`Element`	`id`	Exactamente 36 caracteres.
`Element`	`name`	Entre 1 y 20 caracteres.
`Element`	`description`	Entre 10 y 600 caracteres.
`Element`	`params`	Entre 10 y 600 caracteres.
`Element`	`externalDesc`	Máximo 600 caracteres.
`SendTo`	`id`	Exactamente 36 caracteres.
`SendTo`	`description`	Entre 10 y 600 caracteres.

3.4.3. Restricciones de Conectividad

Regla	Descripción
Sin auto-referencia	El `idRef` de un `SendTo` no puede apuntar al mismo elemento que lo contiene.
Referencia válida	El `idRef` debe resolver a un `Element` existente en el documento.

3.5. Guía de Uso

CIM define conceptualmente una máquina de audio, entendiendo esta como un módulo que puede trabajar en solitario o bien en conjunto con otras máquinas (véase el documento de relaciones entre máquinas CIM). Proporciona el marco para que los diseñadores de sonido establezcan las capacidades de entrada/salida y el flujo interno sin preocuparse por la implementación técnica subyacente. Los modelos resultantes son la base para generar las especificaciones PIM detalladas.

3.6. Manual de Referencia Técnica MDA-Audio-CIM-Relations-Machines

Este documento especifica el lenguaje **MDA-Audio-CIM-Relations-Machines** de relaciones entre máquinas definidas en el documento de especificación de máquinas CIM. Este documento trabaja al mismo nivel de abstracción (CIM), orquestando la comunicación de alto nivel entre máquinas independientes a escala de proyecto.

En el CIM no se tiene por qué hablar de implementaciones concretas, sino de **elementos** abstractos que capturan la intención del diseño de síntesis de la máquina.

3.7. Estructura del Modelo

El documento de relaciones define cómo fluye la información entre máquinas mediante una colección de vínculos abstractos.

3.7.1. Modelo Raíz (`Document`)

El objeto raíz contiene la descripción global del sistema de interconexiones.

Atributo	Tipo	Descripción	Restricciones
`description`	STRING	Propósito general de las interconexiones.	Opcional. 1 a 1000 car.
`relations`	ARRAY[Relation]	Lista de objetos de relación entre máquinas.	Obligatorio.

Ejemplo de Documento:

```
{
  "description": "Simple Lead to Mixer routing.",
  "relations": [ ... ]
}
```

3.7.2. Objeto de Relación (`Relation`)

Define un vínculo direccional entre dos máquinas CIM independientes.

Atributo	Tipo	Descripción	Restricciones
`id`	STRING	Identificador único de la relación.	Obligatorio. 36 car.
`source`	STRING	ID de la máquina de origen.	Obligatorio. 36 car.
`destination`	STRING	ID de la máquina de destino.	Obligatorio. 36 car.
`description`	STRING	Justificación técnica del vínculo.	Obligatorio. 10 a 1000 car.

Ejemplo de Relación:

```
{
  "id": "111e8400-e29b-41d4-a716-446655440111",
  "source": "LeadSynth",
  "destination": "MainMixer",
  "description": "Routes audio from lead to master mixer."
}
```

3.8. Reglas de Validación

Para garantizar la integridad del flujo entre máquinas, se aplican las siguientes restricciones:

3.8.1. Identidad y Formato

- **Unicidad de IDs:** Cada relación en el array `relations` debe tener un `id` único.
- **Formato UUID:** El campo `id` debe cumplir con el estándar de 36 caracteres.

3.8.2. Restricciones de Texto

- **Descripción Global:** Máximo 1000 caracteres.
- **Descripción de Relación:** Entre 10 y 1000 caracteres.

3.8.3. Interfaces Expuestas y Existencia

- **Existencia:** Los IDs especificados en `source` y `destination` deben corresponder a máquinas CIM existentes en el proyecto.

- **Sin Auto-Relación:** Una máquina no puede conectarse consigo misma. Los campos `source` y `destination` deben ser diferentes.
- **Configuración de Origen (`source`):** La máquina referenciada como **origen** DEBE contener al menos un elemento interno configurado con `hasExternalOutput: true`.
- **Configuración de Destino (`destination`):** La máquina referenciada como **destino** DEBE contener al menos un elemento interno configurado con `hasExternalInput: true`.

En resumen, las relaciones CIM solo pueden existir si las interfaces individuales de las máquinas están habilitadas para la comunicación externa y si ambas entidades son máquinas distintas y existentes.

3.9. Guía de Uso

Las relaciones CIM expresan dependencias conceptuales (ej.: *La máquina de síntesis de leads alimenta a la máquina de mezcla principal*). No entran en detalles técnicos de canales, puertos específicos o parámetros muy concretos, ya que esa granularidad se define exclusivamente en el nivel **PIM**.

CONSEJO

Antes de definir una relación, asegúrese de que la máquina origen expone salidas y la destino acepta entradas en sus respectivos modelos CIM. De lo contrario, el validador bloqueará la creación del flujo.

Capítulo 4

PIM (Platform Independent Model)

4.1. Manual de Referencia Técnica MDA-Audio-PIM

Este documento constituye la especificación técnica completa del lenguaje **MDA-Audio-PIM-Machines**. Define todos los nodos, parámetros, reglas de conectividad y restricciones de validación que rigen la gramática. Hay que ver el modelo que especifica este lenguaje como una concretización del modelo CIM, ya con nomenclatura tecnológica orientada a la implementación del sistema de sonido, pero sin llegar a atarse a ningún lenguaje de programación o tecnología específica. Una máquina PIM no es más que una concretización de N máquinas CIM, por lo que es fundamental que el modelo PIM sea coherente con el modelo CIM del que se deriva.

4.2. Bases del Modelo

El lenguaje MDA-Audio-PIM-Machine define un grafo dirigido de audio y control. Todos los elementos del grafo (nodos, aristas y parámetros) comparten una estructura base.

4.2.1. Modelo Raíz (`Model`)

El objeto raíz de un archivo MDA-Audio-PIM-Machine contiene la identificación global y los contenedores de nodos y aristas.

Atributo	Tipo	Descripción	Restricciones
`id`	STRING	Identificador único de la máquina PIM.	Obligatorio. UUIDv4.
`name`	STRING	Nombre descriptivo.	Máximo 20 car.
`description`	STRING	Información general.	Máximo 600 car.
`ids_cim_ref`	ARRAY	Lista de IDs de máquinas CIM vinculadas.	Obligatorio.
`nodes`	ARRAY[Node]	Nodos de audio y control.	Obligatorio.
`edges`	ARRAY[Edge]	Contenedor de conexiones (aristas).	Obligatorio.

Ejemplo de Estructura Raíz:

```
{
  "id": "550e8400-e29b-41d4-a716-446655440001",
  "name": "Full Technical Test",
  "description": "Descripcion de hasta 600 caracteres...",
```

```
  "ids_cim_reference": ["550e8400-e29b-41d4-a716-446655440000"],
  "nodes": [...],
  "edges": [...]
}
```

4.2.2. Interfaces Base

Elementos Principales (Nodos y Aristas)

Atributo	Tipo	Descripción	Restricciones
`id`	STRING	Identificador único universal.	Obligatorio. UUIDv4.
`description`	STRING	Información textual del propósito.	Opcional. Máx 600 car.
`ids_references`	ARRAY	Referencias a elementos del modelo CIM.	Obligatorio.

Estructura de Parámetro de Configuración (`Parameter`)

Cualquier propiedad de configuración de un nodo (excepto el listado `others`) es un objeto de tipo `Parameter`.

- `id`: Identificador único del parámetro (UUID).
- `ids_references`: Referencias a elementos del modelo CIM.
- `initialValue`: Valor inicial del parámetro (tipo *any*).
- `isModifiable`: Indica si acepta modulaciones de otros nodos. Siempre `true` por defecto.
- `isExternalInput`: Flag de externalización. Solo puede ser `true` si `isModifiable` es `true`.
- `description`: Descripción opcional del propósito.

Regla de Integridad Semántica

Un parámetro que no sea modulable internamente (`isModifiable: false`) **no puede** ser una entrada externa (`isExternalInput: true`).

Estructura de Parámetro Dinámico (`OthersParameter`)

Todos los nodos disponen de un array `others` para parámetros customizados.

Campo	Tipo	Descripción
`id`	UUID	Identificador único.
`name`	STRING	Obligatorio. 1 a 20 caracteres.
`initialValue`	STRING	Obligatorio. 1 a 100 caracteres.
`isModifiable`	BOOLEAN	Indica si acepta modulaciones.
`isExternalInput`	BOOLEAN	Flag de externalización.

Puntos de Conexión (`ConnectionPoint`)

Puertos de entrada o salida de audio y control de los nodos.

- `id`: Identificador único (UUID).
- `ids_references`: Referencias conceptuales al modelo CIM.
- `isExternalInput/Output`: Determina si la señal puede cruzar los límites de la máquina.

4.3. Definición de Aristas (`Edges`)

Las aristas establecen la conexión entre nodos.

- **Señal `audio`:** Conecta salidas de sonido con entradas de sonido.
- **Señal `modification`:** Conecta salidas de control con parámetros modulables.
- **Cascada de Integridad:** Al desactivar `isModifiable` en un parámetro, el sistema borra automáticamente todas las aristas con ese destino.
- **Prohibición:** No se puede modificar el parámetro `stereo` de ningún nodo.

4.4. Diccionario de Nodos

4.4.1. Generadores de Audio (`AudioGeneratorNode`)

Todos cuentan con el fragmento `AudioOutputFields`: `stereo` (parámetro estructural), `output_1` y `output_2` (si `stereo` es true).

Oscilador (`oscillator`)

Parámetro	Tipo	Rango / Valores	Descripción
`waveform`	STRING/Matrix	`sine, square, sawtooth, triangle, pulse, matrix`	Forma de la onda.
`frequency`	NUMBER	0 - 20,000	Frecuencia en Hz.
`pulseWidth`	NUMBER	0 - 1	Ancho de pulso (para `pulse`).
`gain`	NUMBER	0 - 1	Ganancia inicial.
`phase`	NUMBER	0 - 2π (6.283)	Fase inicial.
`pan`	NUMBER	-1 a 1	Posicionamiento estéreo.

Ruido (`noise`)

Parámetro	Tipo	Rango / Valores	Descripción
`noiseType`	STRING	`white, pink, brownian`	Color del ruido.
`amplitude`	NUMBER	0 - 1	Amplitud de la señal.
`gain`	NUMBER	0 - 1	Ganancia de salida.
`pan`	NUMBER	-1 a 1	Posicionamiento estéreo.

Sample (`sample`)

Parámetro	Tipo	Rango / Valores	Descripción
`file`	STRING	Ruta de archivo	Path al recurso de audio.
`loop`	BOOLEAN	`true, false`	Indica si el sample se repite.
`gain`	NUMBER	0 - 1	Ganancia de salida.
`pan`	NUMBER	-1 a 1	Paneo (0 si `stereo` es false).

4.4.2. Modificadores de Parámetro (`ParameterModifierNode`)

LFO (`lfo`)

Parámetro	Tipo	Rango / Valores	Descripción
`waveform`	STRING/Matrix	`sine, square, sawtooth, triangle, pulse, matrix`	Onda del modulador.
`rate`	NUMBER	0.01 - 50	Velocidad en Hz.
`amplitude`	NUMBER	0 - 1	Profundidad modulación.
`phase`	NUMBER	0 - 360	Fase en grados.
`sync`	BOOLEAN	`true, false`	Sincro con el tempo.

Envolvente (`envelope`)

Parámetro	Tipo	Rango / Valores	Descripción
`envType`	STRING	`ADSR, ADR, AR, DAHDSR`	Arquitectura envolvente.
`attack`	NUMBER	≥ 0	Tiempo de ataque.
`decay`	NUMBER	≥ 0	Tiempo de caída.
`sustain`	NUMBER	0 - 1	Nivel de sostenido.
`release`	NUMBER	≥ 0	Tiempo de relajación.
`delay`	NUMBER	≥ 0	Retraso (DAHDSR).
`hold`	NUMBER	≥ 0	Mantenimiento (DAHDSR).
`curve`	STRING	`linear, exponential, logarithmic`	Curva interpolación.

4.4.3. Modificadores de Sonido / Efectos (`SoundModifierNode`)

Filtro de Frecuencia (`frequency_filter`)

Parámetro	Tipo	Rango / Valores	Descripción
`filterType`	STRING	`LPF, HPF, BPF, Notch`	Tipo de filtro.
`cutoff`	NUMBER	20 - 20,000	Frecuencia de corte (Hz).
`resonance`	NUMBER	0 - 10	Resonancia (Q).
`slope`	STRING	`12dB, 24dB, 48dB`	Pendiente atenuación.

Reverb (`reverb`)

Parámetro	Tipo	Rango / Valores	Descripción
`roomSize`	NUMBER	0 - 1	Tamaño del espacio.
`damping`	NUMBER	0 - 1	Absorción altas frec.
`decayTime`	NUMBER	≥ 0	Tiempo reverberación.
`dryWet`	NUMBER	0 - 1	Mezcla señal seca/procesada.

Delay / Eco (`delay`)

Parámetro	Tipo	Rango / Valores	Descripción
`delayTime`	NUMBER	≥ 0	Tiempo de retraso.
`feedback`	NUMBER	0 - 1	Retroalimentación.
`lpCutoff`	NUMBER	20 - 20,000	Filtro paso bajo interno.
`hpCutoff`	NUMBER	20 - 20,000	Filtro paso alto interno.
`dryWet`	NUMBER	0 - 1	Mezcla señal seca/procesada.

Distorsión (`distortion`)

Parámetro	Tipo	Rango / Valores	Descripción
`drive`	NUMBER	0 - 1	Cantidad saturación.
`tone`	NUMBER	0 - 1	Control de brillo.
`distType`	STRING	`soft, hard, bitcrush`	Algoritmo distorsión.
`outLevel`	NUMBER	0 - 1	Volumen final salida.

Chorus / Flanger (`chorus_flanger`)

Parámetro	Tipo	Rango / Valores	Descripción
`rate`	NUMBER	≥ 0	Velocidad modulación.
`depth`	NUMBER	0 - 1	Profundidad efecto.
`feedback`	NUMBER	0 - 1	Retroalimentación.
`mix`	NUMBER	0 - 1	Mezcla final.

Compresor (`compressor`)

Parámetro	Tipo	Rango / Valores	Descripción
`threshold`	NUMBER	Valor en dB	Umbral de activación.
`ratio`	NUMBER	≥ 1	Relación compresión.
`attack`	NUMBER	≥ 0	Rapidez ataque.
`release`	NUMBER	≥ 0	Tiempo relajación.
`makeup`	NUMBER	≥ 0	Compensación volumen.

Ecualizador Paramétrico (`equalizer`)

Parámetro	Tipo	Rango / Valores	Descripción
`bandFreq`	NUMBER	20 - 20,000	Frecuencia central banda.
`bandwidth`	NUMBER	Valor Q	Ancho de banda.
`gain`	NUMBER	Valor dB	Realce o atenuación.

4.4.4. Nodos de Mezcla y Nivel

Mezclador (`mixer`)

Suma dinámica de entre 2 y 10 señales de audio. Entradas `input_1` a `input_10`.

Ganancia y Paneo (`gain_pan`)

Control de amplitud y posición. Entrada fija `input_1`. `pan` de -1 a 1.

4.5. Apéndice Técnico de Restricciones

4.5.1. Formato de Matriz (`Matrix`)

- **Tamaño:** Variable entre 4x4 y 1000x1000 elementos.
- **Estructura:** Estrictamente cuadrada (filas == columnas).
- **Contenido:** Solo valores binarios (0 o 1).

4.6. Manual de Referencia Técnica MDA-Audio-PIM-Relations-Machines

Este lenguaje, **MDA-Audio-PIM-Relations-Machines**, permite orquestar las interconexiones entre máquinas PIM. A diferencia del nivel CIM, donde las relaciones son entre máquinas completas, el nivel PIM establece vínculos directos entre puntos de conexión y parámetros específicos.

4.7. Estructura del Documento

El lenguaje utiliza una estructura JSON para definir el grafo de relaciones.

4.7.1. Propiedades Raíz

Atributo	Tipo	Descripción	Restricciones
`description`	STRING	Propósito de la red de relaciones.	Opcional. 1 a 600 car.
`relations`	ARRAY	Lista de objetos de relación detallada.	Obligatorio. Puede estar vacío.

Ejemplo de Estructura Raíz:

```
{
  "description": "0123456789 (hasta 600 caracteres)",
  "relations": [
    {
      "id": "550e8400-e29b-41d4-a716-446655661001",
      "source": "550e8400-e29b-41d4-a716-446655445001",
      "destination": "550e8400-e29b-41d4-a716-446655444004",
      "description": "Modulacion de amplitud (10 a 300
   caracteres)"
    }
  ]
}
```

4.7.2. Objeto Relation (Relación PIM)

Define un vínculo técnico entre un punto de salida y un punto de entrada o parámetro.

Atributo	Tipo	Descripción	Restricciones
`id`	STRING	Identificador único de la relación.	Obligatorio. UUID de 36 car.
`source`	STRING	ID del **output** de una máquina PIM.	Obligatorio. Debe ser externo.
`destination`	STRING	ID del **input** o **parámetro**.	Obligatorio. Debe ser externo.
`description`	STRING	Descripción de la modulación o flujo.	Obligatorio. 10 a 300 car.

Ejemplo de Relación Detallada:

```
{
  "id": "550e8400-e29b-41d4-a716-446655661001",
  "source": "550e8400-e29b-41d4-a716-446655445001",
  "destination": "550e8400-e29b-41d4-a716-446655444004",
  "description": "Conexion de oscilador a ganancia del
   amplificador."
}
```

4.8. Reglas de Validación

El compilador de PIM Relations aplica las siguientes reglas de integridad:

1. **Formato UUID Estricto:** Los campos `id`, `source` y `destination` deben tener exactamente 36 caracteres. No se permiten nombres descriptivos; solo identificadores técnicos.
2. **Unicidad de IDs:** El `id` de cada relación debe ser único en todo el documento.
3. **Restricciones de Texto:**
   - Descripción del documento: máx. 600 caracteres.
   - Descripción de relación: entre 10 y 300 caracteres.

4.9. Guía de Uso Táctico

Este lenguaje es el puente final antes de la generación de código de bajo nivel. Mientras que en *CIM Relations* decimos *"La Máquina A se conecta a la B"*, en *PIM Relations* especificamos *"El output_1 de la instancia 550e8400... se conecta al frequency de la instancia a1b2c3d4..."*.

IMPORTANTE

Los IDs utilizados en `source` y `destination` deben corresponder a identificadores definidos en los archivos `.json` de máquinas PIM correspondientes, siguiendo las reglas ya expuestas.

Capítulo 5

Generación de Paquetes para IA

La aplicación *MDAA* centraliza toda la información de los *DSLs* y los ficheros `JSON` que implementan las máquinas en sus dos niveles (*CIM* y *PIM*). El objetivo final es consolidar este conocimiento en un paquete usable por una Inteligencia Artificial para la generación o análisis de código de audio.

5.1. Proceso de Montaje

La aplicación escanea el proyecto buscando las definiciones de máquinas y sus relaciones. Utilizando la información de los dos niveles:

1. Extrae la estructura funcional del *CIM*.
2. Extrae la especificación técnica del *PIM*.
3. Relaciona ambos niveles para crear un contexto completo del diseño.

5.2. Plantillas y README.md

Para que la IA pueda interpretar correctamente el proyecto, se genera un fichero `README.md` estructurado que actúa como guía. Este fichero se construye siguiendo plantillas predefinidas que organizan la información de la siguiente manera:

- **Visión General**: Resumen del proyecto y su propósito.
- **Arquitectura CIM**: Descripción de las máquinas conceptuales y sus vínculos.
- **Detalle PIM**: Especificación técnica de los nodos y parámetros para cada máquina.

- **Instrucciones de Generación**: Guía para que la IA sepa cómo traducir este modelo a lenguajes como *Pure Data* o *C++*.

El resultado es un paquete comprimido que contiene los esquemas `JSON` y el `README.md` maestro, listo para ser procesado por cualquier modelo de lenguaje avanzado.

Capítulo 6

Ejecución y fuentes de los DSL

En este capítulo se recopila la información de ejecución y validación para cada uno de los lenguajes de dominio específico desarrollados. Esta información se extrae directamente de los archivos `README.md` de cada proyecto DSL, que se encuentran en sus respectivos directorios, `._CODIGO/dsl/[directorio del DSL concreto]/README.md` .

Todos los DSL han sido desarrollados con el *Langium*, con miras a ser implementados en JSON. Los objetos a validar que le pasemos a los scripts de los DSL deberán estar en formato JSON, siguiendo las reglas establecidas por las respectivas gramáticas.

6.1. DSL CIM Machine

Su código fuente se encuentra en `._CODIGO/dsl/mdaa-cim-machine`.

6.1.1. Comandos Principales

- `npm run langium:generate`: Genera el código del lenguaje a partir de la gramática.
- `npm run build`: Compila los servicios y validadores.

6.1.2. Validación de Modelos JSON

Se utiliza el script `validate.ts` para asegurar que los modelos cumplen tanto con la estructura `JSON` como con las reglas del lenguaje.

```
# Validar un fichero individual
node validate.ts examples/valid/valid-1.json

# Validar todos los ficheros de un directorio
node validate.ts examples/valid
```

6.2. DSL CIM Relations

Su código fuente se encuentra en `._CODIGO/dsl/mdaa-cim-relations-machines`.

6.2.1. Comandos Principales

- `npm run langium:generate`: Genera el código del lenguaje.
- `npm run build`: Compila los servicios de validación.

6.2.2. Validación de Modelos JSON

Soporta validación interna y validación cruzada referenciando máquinas reales.

```
# Validacion Interna
node validate.ts examples/valid

# Validacion Cruzada
node validate.ts examples/valid examples/machines
```

6.3. DSL PIM Machine

Su código fuente se encuentra en `._CODIGO/dsl/mdaa-pim-machine`.

6.3.1. Comandos Principales

- `npm run langium:generate`: Genera el código TypeScript a partir de la gramática.
- `npm run build`: Compila el proyecto.

6.3.2. Validación de Modelos JSON

```
# Validacion individual o directorio
node validate.ts examples/valid

# Validacion Cruzada (PIM + Universo CIM)
node validate.ts examples/valid examples/machines
```

6.4. DSL PIM Relations

Su código fuente se encuentra en `._CODIGO/dsl/mdaa-pim-relations-machines`.

6.4.1. Validación

Soporta dos modos: estructural (sintáctica) y cruzada (semántica).

```
# Validacion Estructural
node validate.ts <fichero_o_directorio>

# Validacion Cruzada
node validate.ts <fichero_o_directorio> examples/machines
```